COMP1011 Individual Project

Habit Tracker – A Command-Line Habit Tracking System

Submitted by: YIMI Supphakon

1. Project Objectives

The Habit Tracker is a command-line interface (CLI) application written in C++ that allows users to build and maintain positive daily habits. The program targets students and individuals who prefer a lightweight, terminal-based tool with no external dependencies beyond the C++ standard library.

The key objectives of this project are:

- Allow users to define, add, and delete personal habits.
- Enable daily habit check-in, allowing users to mark habits as completed or not for any given date.
- Persist tracking data across sessions using a CSV file (habits.csv), so progress is never lost.
- Provide a heat-map calendar visualization in the terminal, giving users a quick visual overview of their consistency over any given month.
- Present a personal dashboard with streak statistics and completion summaries.

2. Key Functions

This section describes five representative functions developed for the project, covering date logic, file management, data search, UI rendering, and validation.

2.1 Heat-Map Calendar Visualization: PrintCalendar()

Function signature:

```
void PrintCalendar(string month, string year);
```

Design and Purpose:

PrintCalendar() renders a month view of a habit calendar directly in the terminal. Each day is coloured using ANSI escape codes to reflect how many habits were completed on that day relative to the total number of habits. This provides a heat map of the user's consistency.

Algorithm and Data Structures:

- The function retrieves the first weekday of the month by calling getDayOfWeek(DateToStruct(year + "-" + month + "-01")).
- It prints blank cells for the offset days before day 1, ensuring correct column alignment (Sun–Sat header).

- For each day, it looks up progressBydate[now_date] (an unordered_map<string, int> storing the count of completed habits per date) and computes the completion ratio.
- HeatColor(completed, total) maps five ratio bands (0%, 0-25%, 25-50%, 50-75%, 75-100%) to five distinct ANSI background colours using RGB escape sequences.
- An explanation of the colors is printed below the calendar to decode the colour scale.

The key data structure is unordered_map<string, int> progressBydate, calculated during file read-in, which gives O(1) lookup per day.

2.2 Efficient Date Lookup: BinarySearchDate()

Function signature:

```
bool BinarySearchDate(string input_date);
```

Design and Purpose:

BinarySearchDate() determines whether a specific date exists in the current dataset. Variable dates are stored in a sorted vector<pair<string, int>> (string of date + its integer value for comparisons), so I can use binary search, which runs in O(log n) time faster than O(n) linear scan.

Algorithm:

- Each date string is converted to an integer via DateToNum(), which encodes year, month, and day into a single comparable number (year × 366 + month × 31 + day).
- The variable dates is maintained in ascending order by this integer value.
- The function applies the classic two-pointer binary search: l = 0, r = dates.size() - 1, narrowing the range until l == r.
- If dates[l].second equals the target integer, the date exists; otherwise, it does not.

This function is called before every write operation to avoid duplicate date entries and before user tracking to verify the date was previously logged.

2.3 CSV Persistence Layer: ReadData() / SaveData()

Function signatures:

```
void ReadData();
void SaveData();
```

Design and Purpose:

These two functions implement the complete persistence layer of the application. All habit names and tracking records are stored in habits.csv, a plain-text comma-separated file. This keeps data human-readable and easily exportable.

ReadData() – Algorithm:

- Opens habits.csv for reading; if the file does not exist, the function exits gracefully and the app starts with an empty state.
- Parses the header row with a stringstream to extract all habit names into the habits vector.
- For every subsequent row, extracts the date string, pushes it into dates, and reads one boolean value per habit column into tracking_data[habit][date].
- Simultaneously accumulates progressBydate[date] by counting columns with value '1'.

SaveData() – Algorithm:

- Reopens habits.csv in overwrite mode (ios::out).
- Writes the header row (date, followed by habit names).
- Sorts dates by their integer representation using a lambda comparator before writing, ensuring chronological order regardless of insertion order.
- Writes one row per date, with 1 or 0 for each habit.

The key data structure is unordered_map<string, unordered_map<string, bool>> tracking_data, providing O(1) access to any habit–date pair.

2.4 Date Validation: IsValidDate()

Function signature:

```
bool IsValidDate(string input_date);
```

Design and Purpose:

IsValidDate() guards all user date inputs against malformed or out-of-range values. Accepting invalid dates would corrupt the CSV and break sorting or calendar rendering.

Validation Steps:

- Character check: rejects any character that is not a digit or hyphen '-'.
- Structural parse: calls DateToStruct() to split the string into {year, month, day} components.

- Range checks: year must be 4 digits and >= 1000; month must be 1–12; day must be 1–31.
- Future-date guard: converts both the input and the current system date to integers via DateToNum() and rejects any date greater than today.

3. Examples and User Manual

3.1 Launching the Application

Compile and run the program:

```
g++ -o habit_tracker main.cpp date.cpp fileManagement.cpp screen.cpp
./habit_tracker
```

On first launch, if no habits.csv file is found, the application starts with an empty habit list. The main menu is displayed immediately.

Note: if `./habit_tracker` *does not work, try* `habit_tracker` or `habit_tracker.exe`

3.2 Main Menu

The main menu presents five options:

Option	Description
[1] Track Today	Mark habits as done or not for the current date.
[2] Track The Day	Mark habits for a past date entered by the user.
[3] Manage Habits	Add or delete habit names from the tracked list.
[4] Dashboard	View streak statistics and the heat-map calendar.
[5] Exit	Quit the program.

3.3 Sample Interaction – Track Today

When the user selects option [1], the application displays:

The user types a habit number to toggle its completion status. Entering 'q' returns to the main menu.

3.4 Sample Interaction – Heat-Map Calendar

When the user selects option [4] and navigates to the calendar view, a month calendar is rendered in colour. Each date cell is shaded from light teal (few habits completed) to dark teal (all habits completed), with gray representing no data.

Below the calendar, a legend is printed:

(The shaded boxes above appear as colour gradients in the actual terminal output — screenshots are provided in Section 6.)

3.5 habits.csv – File Format Example

Each row corresponds to one date. A value of 1 means the habit was completed; 0 means it was not.

4. Data Structures Summary

Variable	Type	Purpose
habits	vector<string>	Ordered list of habit names (column order in CSV).
dates	vector<pair<string,int>>	All tracked dates with integer key for sorting and binary search.
tracking_data	unordered_map<string, unordered_map<string,bool>>	Two-level map: habit name → date → completion status.
progressBydate	unordered_map<string,int>	Number of habits completed per date; drives heat-map colour.

5. AI Tool Disclosure

In accordance with the project requirements, the following disclosure is made regarding the use of AI-generated code:

- The function getDayOfWeek() in date.cpp was developed with the assistance of an AI tool. This function implements Zeller's Congruence to compute the day of the week for a given date. The generated code is clearly marked in the source file with the comment // GenAI code ... // -----.
- The function ClearPreviousLines() in screen.cpp, which uses ANSI escape sequences to erase terminal lines, was also generated with AI assistance and is similarly marked in the source.
- All other functions, including the CSV persistence layer, binary search, heat-map rendering, date arithmetic, and the overall program architecture, were written independently by the author.

No open-source projects were used as a base. The AI-generated segments are self-contained utility functions that do not constitute the core logic of the application.

6. Screenshots of Key Results

The following screenshots demonstrate the application running in a terminal environment. Please append actual screenshots below each caption.

[Screenshot 1] Main menu as displayed on launch.

[Screenshot 2] Track Today screen, showing habit toggle interface.

[Screenshot 3] Heat-map calendar view for a sample month.

[Screenshot 4] Personal Dashboard with streak statistics.

[Screenshot 5] habits.csv file content after saving data.